

Contrôle de concurrence en SQL avec Oracle

G. RASCHIA

Dernière modification : 1^{er} avril 2026

Objectif du TP :

- comprendre les mécanismes du contrôle de concurrence ;
- reproduire les anomalies courantes ;
- expérimenter les niveaux d'isolation ANSI SQL.

Préambule

La norme ANSI SQL définit quelques commandes pour la gestion des transactions (`COMMIT`, `SAVEPOINT`, `ROLLBACK`, `SET TRANSACTION`), regroupées sous le terme de TCL (*Transaction Control Language*), et distinctes des instructions *Data Definition Language*¹ (DDL), *Data Manipulation Language*² (DML), ou encore *Data Control Language*³ (DCL).

Les commandes TCL permettent un contrôle fin de la concurrence.

Conformément à la norme SQL2, on peut définir le *mode*—lecture seule ou lecture écriture—d'une transaction par l'instruction :

```
SET TRANSACTION { READ ONLY | READ WRITE };
```

Il est également possible de spécifier le *niveau d'isolation* d'une transaction :

```
SET TRANSACTION ISOLATION LEVEL { SERIALIZABLE | REPEATABLE READ |  
                                  READ COMMITTED | READ UNCOMMITTED };
```

Dans la suite, nous allons déterminer expérimentalement la signification des différents niveaux d'isolation par l'observation et l'analyse des anomalies qu'ils présentent.

1 Prise d'information

La réalisation du TP requiert un SGBD SQL muni des commandes TCL. Il s'agit de Oracle Database 21c. Le serveur de l'École est accessible aux coordonnées :

```
user          = {n°étudiant}  
password      = {n°étudiant}  
host          = 172.26.71.202  
port          = 1521  
service name  = orclpdb1
```

Nous recommandons en outre le client `SQLDeveloper` pour s'y connecter. Il est impératif de s'assurer que le mode « validation automatique » (`autocommit`) du client SQL choisi soit débrayé.

1. `CREATE`, `DROP`, `ALTER`, `RENAME`, `TRUNCATE`.

2. `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

3. `GRANT`, `REVOKE`.

Les réponses aux questions de cette section sont à chercher dans la [documentation officielle de Oracle Database 21](#).

1. Déterminer le mode (parmi `READ WRITE` ou `READ ONLY`) et le niveau d'isolation *par défaut* d'une transaction.
2. Parmi les quatre degrés d'isolation du ANSI/SQL, lesquels sont effectivement implémentés dans Oracle ?
3. Identifier précisément les mécanismes transactionnels, *explicites* ou *implicites*, en répondant aux questions :
 - Quelles instructions déclenchent une nouvelle transaction ? En particulier, comment Oracle traite l'instruction `SELECT` ?
 - Sous quelles conditions se termine une transaction ?

Remerciements

Cet énoncé fut initialement proposé par Cédric LHOSSAINE dans le module « Bases de données avancées » des Formations en Informatique de Lille à l'université Lille 1. Il a été par la suite significativement remanié.

2 Échauffement

Les conditions d'observation du contrôle de concurrence requièrent l'ouverture de deux sessions simultanées, simulant deux applications distinctes réalisant des exécutions concurrentes de transactions. Il est donc possible de travailler en binôme d'étudiants (x et y) connectés simultanément (session n°1 et session n°2) sur la même base de données. Il est également tout à fait possible de travailler seul·e, et il faut dans ce cas ouvrir deux connexions distinctes.

Les manipulations qui suivent concernent une table unique t ayant 2 colonnes numériques a et b . On utilisera les abbréviations suivantes :

- `select t` pour `select * from t`;
- `select (a=1)` pour `select * from t where a=1`;
- `insert (3,6)` pour `insert into t values (3,6)`;
- `update (a<-a+1)` pour `update t set a=a+1`;
- `update (a<-3)|(a=4)` pour `update t set a=3 where a=4`;
- `delete (a=4)` pour `delete from t where a=4`;

1. Créer la table t – une seule par binôme (ou expérience de concurrence) – et, si nécessaire, la rendre accessible aux deux utilisateurs :

session n°1 (x)	session n°2 (y)
01 <code>create table t(a number, b number);</code>	
02 <code>grant select, insert, delete, update</code>	
03 <code>on t to y;</code>	
	<code>create synonym t for x.t;</code>

2. Ensuite, insérer les lignes (0,1), (1,3) et (2,3) et valider (`commit`).

3 Expériences avec les transactions

Il est demandé de réaliser et d'*analyser* tous les scénarios proposés, pour justifier la réponse apportée à chacune des questions. En pratique, vous pouvez produire pour chaque question, un tableau comme celui-ci, complété par une brève analyse/réponse textuelle à la ou les question(s) posée(s) :

session n°1	session n°2	observation	commentaire
01	<code>select t</code>	a	b
		0	0
		2	3
		0	1
02	<code>update (b<-42) (a=1)</code>	1 row updated.	Il se produit ceci...
03	...	...	...

Il est également possible de prolonger le scénario proposé pour approfondir l'analyse.

Pour toutes les expériences à venir, et sauf indication contraire, les transactions s'exécutent en mode `READ WRITE` et au niveau d'isolation `READ COMMITTED`.

En outre, la table *t* doit être dans son état original *au début de chaque expérience*.

- Déterminer les deux anomalies mises en évidence par l'expérience suivante, l'une empêchée l'autre survenue :

session n°1	session n°2
01 <code>select t</code>	
02	<code>update (a<-a+1)</code>
03 <code>select t</code>	
04	<code>commit</code>
05 <code>select t</code>	

- Soit l'histoire suivante :

session n°1	session n°2
01 <code>select (b=3)</code>	
02	<code>insert (3,3)</code>
03	<code>commit</code>
04 <code>select (b=3)</code>	
05	<code>delete (b=3)</code>
06	<code>commit</code>
07 <code>select (b=3)</code>	

- Déterminer l'anomalie mise en évidence par l'expérience.
 - Rejouer l'histoire au niveau d'isolation sérialisable dans la session n°1.
- Soit l'histoire suivante :

session n°1	session n°2
01	<code>select (a=2)</code>
02 <code>select (a=2)</code>	
03	<code>update (b<-22) (a=2)</code>
04	<code>commit</code>
05 <code>update (b<-11) (a=2)</code>	
06 <code>commit</code>	

- (a) Donner la preuve que cette histoire n'est pas sérialisable. De quelle anomalie s'agit-il ?
 (b) Rejouer l'histoire en déclarant sérialisable la transaction de la session n°1.

4. Soit l'histoire suivante :

	session n°1	session n°2
01		<code>update (a<-a+1)</code>
02		<code>select t</code>
03	<code>select t</code>	
04	<code>insert (3,0)</code>	
05	<code>delete (a=2)</code>	
06		<code>commit</code>
07	<code>commit</code>	
08	<code>select t</code>	

- (a) Déterminer le problème mis en évidence, et sa résolution par Oracle.
 (b) Rejouer l'histoire en déclarant l'attribut `a` clé primaire. Observer son déroulement, successivement avec `commit` et `rollback` dans la session n°2.

5. Soit l'histoire suivante :

	session n°1	session n°2
01	<code>update (b<-7) (a=2)</code>	
02		<code>update (b<-42) (a=1)</code>
03		<code>select (a=2)</code>
04		<code>update (b<-42) (a=2)</code>
05	<code>update (b<-7) (a=1)</code>	
06		<code>commit</code>
07	<code>commit</code>	
08	<code>select t</code>	

- (a) Dans ce scénario, recenser les verrous posés – types et objets – par x (session n°1) et par y (session n°2). Comment se terminent les transactions ?
 (b) Rejouer ce scénario au niveau d'isolation `serializable`.

6. Indiquer le problème et rapporter la manière dont il est résolu dans le plan suivant :

	session n°1	session n°2
01	<code>set transaction read only</code>	
02	<code>select t</code>	
03		<code>update (a<-a+1)</code>
04		<code>commit</code>
05	<code>select t</code>	
06		<code>update (a<-a+1)</code>
07		<code>commit</code>
08	<code>select t</code>	
09	<code>commit</code>	

7. SQL propose une clause `for update` à placer à la fin de l'instruction `select-from-where`. L'expérience suivante va permettre de comprendre son utilité.

session n°1	session n°2
01	<code>select (b=1)</code>
02 <code>select (b=1) for update</code>	
03	<code>select (b=1)</code>
04	<code>update (b<-10) (b=1)</code>
05 <code>update (b<-42) (b=1)</code>	
06 <code>commit</code>	
07	<code>select t</code>
08	<code>commit</code>

Quel type de verrouillage est opéré dans le plan ci-dessus ?

8. Expliquer les résultats obtenus par l'exécution du plan suivant :

session n°1	session n°2
01 <code>lock table t in share mode</code>	
02	<code>select t</code>
03	<code>update (a<-a+1) (a=2)</code>
04 <code>select t</code>	
05 <code>commit</code>	
06 <code>lock table t in exclusive mode</code>	
07	<code>commit</code>
08	<code>select t</code>
09 <code>update (a<-a+1)</code>	
10 <code>commit</code>	

9. Oracle dispose de « points de sauvegarde intermédiaires », déclarés à l'aide de l'instruction `savepoint`. L'expérience suivante, qui nécessite une troisième session, permet d'en comprendre le fonctionnement.

session n°1	session n°2	session n°3
01 <code>update (b<-4) (a=0)</code>		
02 <code>savepoint after_a0</code>		
03 <code>update (b<-4) (a=2)</code>		
04	<code>update (b<-5) (a=2)</code>	
05 <code>rollback to savepoint after_a0</code>		
06		<code>update (b<-6) (a=2)</code>
07 <code>commit</code>		
08		<code>commit</code>
09	<code>select t</code>	
10	<code>commit</code>	

10. Soit l'histoire suivante, jouée pour chacune des sessions dans le niveau d'isolation sérialisable et soumise à la contrainte de cohérence $(\Sigma b) \leq 8$:

	session n°1	session n°2
01	<code>select sum(b) from t</code>	
02		<code>select sum(b) from t</code>
03	<code>update (b<-b+1) (a=1)</code>	
04		<code>update (b<-b+1) (a=2)</code>
05	<code>commit</code>	
06		<code>commit</code>

En examinant l'état de la base de données, relever le problème mis en évidence et expliquer cette « curiosité ».

4 Conclusion générale

Faire la synthèse de ce qui a été observé, et livrer un commentaire sur les spécificités du contrôle de concurrence dans Oracle vis-à-vis des concepts du cours.